

Arquitetura DDD para integracoes atomicas do simtr-hub

Status e autoridade

- **Status:** aceito
- **Data:** 2026-07-11
- **Estado verificado:** 2026-07-16, incluindo a decisao pragmatica de uso do Quarkus da Fase 10 e o cenario evolutivo de exposicao de capacidades por MCP
- **Escopo:** refatoracao do comportamento atualmente implementado e demonstracao de extensibilidade para novas entradas, sem implementa-las nesta etapa
- **Plano executavel:** ../tasks/plan.md
- **Checklist de retomada:** ../tasks/todo.md
- **Fonte complementar do cenario MCP:**
proposta-arquitetura-capacidades-rest-orquestracao-mcp.md

Este e o documento arquitetural canonico para a refatoracao. Agentes e pessoas devem le-lo antes de alterar packages, portas, DTOs, adapters, tratamento de erros ou testes relacionados.

Fundamentos de organizacao

Organizacao por dominio

Ha duas formas comuns de agrupar o codigo: por tecnologia, reunindo Resources, services, clients e mappers de toda a solucao, ou por dominio, reunindo os artefatos que realizam uma capacidade de negocio. O `simtr-hub` adota **package by domain**.

Assim, `arvoredocumento`, `conformidade`, `dossieproduto` e `gestaodocumento` possuem fronteiras proprias. Os nomes dos sistemas externos e das tecnologias aparecem nas bordas quando fazem parte do contrato, mas nao definem um dominio interno compartilhado. Essa organizacao reduz o acoplamento entre capacidades e torna explicito quem e responsavel por cada operacao.

Codigo da aplicacao e codigo de infraestrutura

O **codigo da aplicacao** representa a logica de negocio e as funcionalidades centrais do software. Ele expressa capacidades, coordena casos de uso e trabalha com portas e modelos internos.

O **codigo de infraestrutura** implementa as tecnologias que apoiam essas funcionalidades. Em termos gerais, pode incluir consultas SQL, requisicoes HTTP, chamadas AMQP, persistencia, mensageria, armazenamento e integracoes externas. No `simtr-hub` atual, inclui principalmente os endpoints REST, as chamadas HTTP ao MTR, a leitura de fixtures do simulador, serializacao, autenticacao, fault tolerance e observabilidade. Uma futura exposicao por MCP tambem pertence a infraestrutura de entrada: protocolo, transporte, schema, autenticacao e serializacao ficam no adapter MCP, e nao no nucleo.

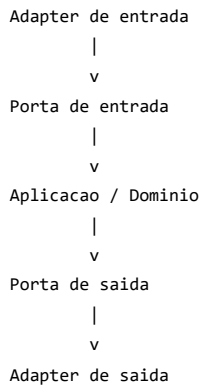
Essa separacao aplica o principio de separacao de responsabilidades e contribui para:

- simplificar os testes;
- reduzir o acoplamento com tecnologias especificas;
- reutilizar a logica da aplicacao por entradas diferentes;
- substituir mecanismos de integracao com menor impacto;
- facilitar a manutencao e a identificacao de problemas.

Na solucao atual, essa separacao define responsabilidades; ela nao separa codigo "com Quarkus" de codigo "sem Quarkus". O framework pode apoiar qualquer componente. O que permanece isolado sao as dependencias entre nucleo, bordas e contratos externos.

Arquitetura Hexagonal

A **Arquitetura Hexagonal**, também conhecida como **Portas e Adaptadores**, promove a separação entre o núcleo da aplicação e os detalhes de infraestrutura.



A regra central é que o núcleo expresse capacidades sem depender dos adaptadores que realizam REST, MCP, persistência, mensageria, armazenamento, integrações externas ou simulação. Essa direção de dependência não exige um núcleo livre de framework: Quarkus e suas APIs podem ser usados no domínio, na aplicação, nas portas e nos casos de uso. As dependências estruturais continuam apontando das bordas para as portas e os modelos do núcleo.

Portas

Uma **porta** é um contrato orientado a uma capacidade. Ela pode usar Quarkus ou outra API de framework quando isso for útil, sem deixar de representar a linguagem da aplicação. O que ela não deve expor por conveniência são DTOs e detalhes exclusivos de uma borda externa.

- A **porta de entrada** representa uma capacidade oferecida pela aplicação. Ela permite que uma requisição HTTP, uma chamada MCP, uma mensagem, uma tarefa agendada ou uma futura etapa de orquestração acione um caso de uso.
- A **porta de saída** representa uma necessidade externa da aplicação. Ela permite consultar ou persistir dados, chamar outro sistema ou publicar uma mensagem sem acoplar o caso de uso ao mecanismo utilizado.

As portas expressam intenções e capacidades, não o protocolo ou o fornecedor que as implementa.

Adaptadores

Um **adapter** conecta uma porta a um mecanismo específico. Adapters de entrada traduzem uma interação externa para uma porta de entrada. Adapters de saída implementam portas de saída e traduzem os tipos internos para os contratos exigidos pela infraestrutura.

Adapters dependem das portas. O núcleo não depende dos adapters, e adapters de tipos diferentes não dependem diretamente uns dos outros.

Direção da dependência e papel do orquestrador

O papel arquitetural de um orquestrador depende da direção da interação; o nome "orquestrador" sozinho não determina sua camada.

Quando um **orquestrador externo inicia uma operação**, ele é um ator de entrada. O protocolo é tratado por um adapter de entrada, que aciona a porta da aplicação:

```
Orquestrador externo
-> adapter de entrada (REST ou mensagem)
    -> porta de entrada
        -> aplicacao
```

Quando um **agente externo usa uma capacidade por MCP**, ele também é um ator de entrada. Uma tool MCP é publicada por um adapter próprio, que traduz seu contrato para a porta de entrada existente:

```
Agente externo
-> cliente MCP
    -> adapter MCP de entrada (futuro)
        -> porta de entrada existente
            -> aplicacao
```

O agente pode coordenar chamadas externamente, mas essa coordenação não transforma MCP em camada de aplicação. A tool continua sendo apenas uma borda de entrada e não chama Resource REST, REST Client MTR nem adapter de saída.

Quando existe um **orquestrador local que coordena casos de uso**, essa coordenação pertence à camada de aplicação. Ele permanece atrás de sua própria porta de entrada, compõe capacidades do mesmo domínio por seus contratos de entrada e atravessa outro domínio somente por uma porta de saída do consumidor e uma camada anticorrupção. Ele não conhece Resources, REST Clients ou adapters:

```
Adapter de entrada
-> porta de entrada do fluxo
    -> orquestrador da aplicacao (futuro)
        |-> porta de entrada de capacidade atomica do mesmo dominio
        `-> porta de saida do consumidor
            -> adapter / camada anticorruptcao
                -> outro dominio
```

Quando a aplicação precisa acionar um **orquestrador externo como dependencia**, a interação pertence ao lado de saída:

```
Aplicacao
-> porta de saida
    -> adapter de saida
        -> orquestrador externo
```

Esses quatro desenhos explicam possibilidades arquiteturais, não componentes existentes. O `simtr-hub` atual não possui orquestrador local, motor de workflow nem adapter dedicado a um orquestrador externo. Também não possui MCP Server nem adapter MCP.

Arquitetura Hexagonal no mundo dos microserviços

Embora tenha "arquitetura" no nome, a Arquitetura Hexagonal é adotada neste documento como um **padrão tático para estruturar o código e os componentes dentro de um serviço específico**.

Ela não define a visão de alto nível, não determina como serviços devem se comunicar e não estabelece a estrutura de um sistema composto por múltiplos serviços. Comunicação por REST, mensageria ou outro mecanismo pertence ao desenho arquitetural do sistema.

No `simtr-hub`, o padrão hexagonal organiza o interior dos domínios e das capacidades. O DDD define os limites de negócio e as formas de colaboração; uma eventual extração para microserviços exige decisões próprias de contratos, comunicação, consistência e operação.

Uso pragmatico

A adocao deve considerar os beneficios obtidos e o custo das abstracoes. O padrao nao e uma solucao universal e nao justifica componentes sem uso real. Portas, adapters e pastas so devem existir quando houver uma capacidade e um consumidor concretos.

Arquitetura Hexagonal nao e adotada como politica de pureza tecnologica. **Quarkus pode ser usado em qualquer componente ou package da solucao e nao deve ser impedido pela arquitetura ou pelo build**, inclusive em dominio, aplicacao, portas e casos de uso. A mesma orientacao vale para APIs associadas, como Jakarta, MicroProfile, Mutiny, Jackson e OpenTelemetry: o simples uso de uma dessas tecnologias nao caracteriza violacao de camada.

Essa liberdade e uma permissao, nao uma obrigacao. Ela tambem nao autoriza o dominio a depender de um Resource ou adapter, a aplicacao a consumir DTO de borda, ou um dominio a acessar internamente outro dominio. Uma regra pode restringir um papel arquitetural especifico — por exemplo, REST Clients continuam confinados ao adapter MTR —, mas nao pode rejeitar uma classe somente porque ela usa Quarkus. Os guardrails protegem responsabilidades e direcao de dependencias, nao pureza de framework.

Visao da solucao

O `simtr-hub` e um monolito modular organizado por dominios de negocio. A solucao implementada oferece oito capacidades atomicas por endpoints REST publicos e integra cada capacidade ao MTR ou ao simulador por adapters de saida independentes. Cada dominio concentra seus casos de uso, modelos internos e contratos de aplicacao; os detalhes exclusivos de protocolo e dos contratos externos permanecem nas bordas. Quarkus, como framework da solucao, pode apoiar qualquer desses componentes.

O fluxo executado atualmente pode ser lido assim:

```
cliente HTTP
-> adapter REST de entrada
    -> porta de entrada
        -> caso de uso atômico
            -> porta de saida
                -> adapter selecionado por configuracao
                    |-- MTR
                    `-- simulador
```

No fluxo atual, a entrada e REST e as duas saidas alternativas sao MTR e simulador. O caso de uso nao conhece Resource, REST Client, URL, DTO MTR, fixture nem o mecanismo de selecao CDI.

A mesma estrutura permite adicionar, no futuro, um adapter MCP de entrada ao lado do adapter REST. Esse adapter reutiliza as portas e os casos de uso existentes sem introduzir MCP no dominio ou na aplicacao. O cenario demonstra uma propriedade da arquitetura; nao afirma que MCP esteja implementado ou aprovado para exposicao operacional.

Nao existe nesta solucao um endpoint unico de pre-validacao, orquestrador local, motor de workflow ou comunicacao distribuida entre os dominios. A composicao por orquestradores e a extracao para microservicos sao possibilidades futuras, explicitamente separadas do estado implementado. A secao de cobertura frente a especificacao identifica os oito endpoints existentes e os cinco que ainda nao existem no Hub.

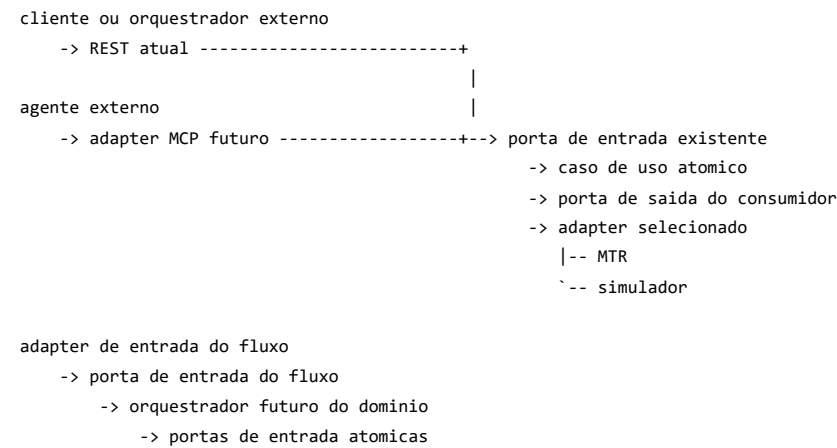
Intencao confirmada

Transformar o `simtr-hub` em um monolito modular DDD, composto por capacidades atomicas e desacopladas. Os endpoints REST atuais e os futuros orquestradores devem ser entradas diferentes para os mesmos casos de uso. Uma futura exposicao MCP deve ser outra entrada para essas mesmas

capacidades, implementada exclusivamente na borda. O núcleo não dependerá de Quarkus Flow, de APIs MCP nem pressuporá que essas tecnologias serão adotadas.

A refatoração deve preparar a futura extração dos módulos em microsserviços sem implementar agora novos endpoints, workflows ou integrações funcionais.

Resultado esperado

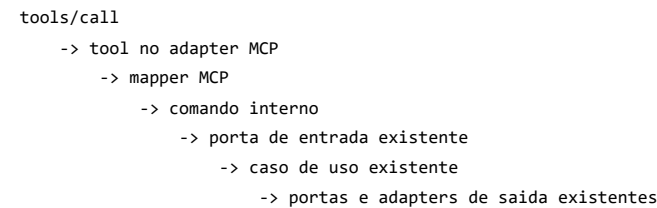


- REST, MCP e orquestradores não conhecem REST Client, DTO MTR, URL ou política HTTP.
- REST e MCP possuem contratos de borda independentes e convergem para as mesmas portas de entrada.
- Um orquestrador do mesmo domínio pode compor capacidades atômicas desse domínio.
- Uma chamada entre domínios usa uma porta de saída do consumidor e uma camada anticorrupção.
- Enquanto os domínios estiverem no mesmo processo, a camada anticorrupção pode usar um adaptador local. Depois da extração, somente esse adaptador muda para HTTP ou mensageria.

Cenário evolutivo: capacidades por MCP

Posicionamento arquitetural

MCP é uma opção futura de protocolo de entrada para agentes externos. Ele não substitui REST, não implementa regras de negócio e não se torna um novo domínio. A extensão preserva a direção de dependência da Arquitetura Hexagonal:



O limite normal da mudança é a criação de componentes em `<domínio>.adaptador.entrada.mcp`, além da configuração técnica necessária para publicar e proteger o protocolo. Não mudam por causa do MCP:

- domínio, regras e modelos de negócio;
- casos de uso e portas de entrada existentes;
- portas de saída e seleção entre MTR e simulador;
- adaptadores MTR e simulador;
- contratos REST públicos e suas políticas de compatibilidade;
- políticas de fault tolerance das integrações de saída.

Se a exposicao de uma tool revelar uma precondicao ausente, a regra deve ser corrigida no caso de uso para proteger igualmente REST, MCP, orquestracao interna e testes. Essa e uma correcao da capacidade de negocio, nao uma regra exclusiva da tool.

Contratos e granularidade

Cada tool deve corresponder inicialmente a uma capacidade atomica e acionar sua porta de entrada. Somente as capacidades com consumidor, autorizacao e risco avaliados recebem tools; nao e necessario expor as oito capacidades de uma vez. Uma futura tool composta deve chamar a porta de entrada de um fluxo da aplicacao, em vez de duplicar a coordenacao no adapter MCP.

```
DTO MCP de entrada
-> mapper MCP
    -> comando interno
        -> porta de entrada
            -> resultado interno
                -> mapper MCP
                    -> DTO MCP de saida
```

DTOs, schemas, nomes de tools e erros MCP sao contratos exclusivos dessa borda. Eles nao reutilizam DTOs REST, MTR ou do simulador e nao atravessam para o nucleo. Isso permite que REST e MCP evoluam independentemente, preservando a semantica da capacidade.

Uma estrutura opcional, criada somente quando houver uma tool concreta, e:

```
<dominio>/adaptador/entrada/mcp/
|-- dto/
|-- mapper/
`-- tool/
```

Seguranca e operacao

A exposicao MCP amplia a superficie de entrada e exige autenticacao de servico, autorizacao por capacidade e propagacao controlada da identidade. Tools mutaveis nao compartilham uma permissao generica irrestrita. Argumentos sensiveis, tokens, credenciais, URLs internas, stacks e DTOs de erro MTR nao podem aparecer em respostas, logs, traces ou memoria de conversacao.

`ObterCredencialContainer` requer decisao de seguranca separada antes de virar tool. A existencia da porta nao implica autorizacao para expor a credencial a um agente.

Spans e logs devem identificar a origem da entrada (REST , MCP ou WORKFLOW_LOCAL) sem alterar os sinais ja considerados contratuais. O trace iniciado na chamada MCP deve continuar pelo caso de uso e pelo adapter MTR, com filtragem de atributos sensiveis.

Consequencias e trade-offs

O beneficio e demonstrar que uma nova forma de consumo pode ser adicionada sem reescrever a capacidade: a variacao fica concentrada em DTOs, mappers, autenticacao e publicacao do adapter de entrada. O custo e uma nova superficie operacional e de seguranca, alem de novos contratos e testes por tool.

Enquanto o MCP Server estiver no mesmo processo, o adapter chama diretamente as portas de entrada. Se uma necessidade futura exigir um servico MCP separado, esse servico passa a ser um consumidor externo dos contratos publicos do Hub, por REST ou mensageria, e deixa de ser um adapter interno. Essa mudanca de implantacao nao autoriza acesso direto ao nucleo entre processos.

Dominios e capacidades atuais

arvoredocumento

É dono da capacidade atômica de consultar o processo parametrizado usado no cálculo futuro da árvore documental.

Capacidade existente:

- ConsultarProcessoParametrizado

O path REST atual deve ser preservado. O package interno `parametrizacao` deixa de representar um domínio. O nome `parametrizacao` pode continuar aparecendo somente na borda de integração com o sistema MTR correspondente.

O cálculo da árvore, os serviços de IA e a orquestração da árvore não fazem parte desta etapa.

conformidade

É dono da capacidade atômica de consultar um checklist por identificador e versão.

Capacidade existente:

- ConsultarChecklist

Essa capacidade não compartilha um grande modelo de parametrização com `arvoredocumento`. Cada domínio possui seu contrato de aplicação, modelos internos e mapeamento do contrato externo.

A análise de documentos e a orquestração de conformidade não fazem parte desta etapa.

dossieproduto

É dono das capacidades atômicas necessárias ao ciclo de vida do dossie no MTR. Nesta etapa não existem aggregate, máquina de estados ou sequenciamento inventados.

Capacidades existentes:

- CriarDossieProduto
- AtualizarFormularioDossieProduto
- IncluirDocumentoDossieProduto
- RegistrarValidacaoNegocialDossieProduto
- IniciarOuAvancarWorkflowDossieProduto

Endpoints adicionais descritos em documentos funcionais, mas não implementados, não entraram na refatoração. A seção seguinte identifica cada um explicitamente; sua eventual implementação exige decisão, contrato, fase e capacidade atômica próprios.

gestaodocumento

É uma capacidade transversal que obtém do MTR a credencial de acesso ao container documental.

Capacidade existente:

- ObterCredencialContainer

O `simtr-hub` não envia arquivos ao Azure, não reutiliza a SAS, não mantém cache da credencial e não fixa sua duração no código. O microserviço consumidor é responsável por reutilizar a credencial até a expiração informada pelo contrato e solicitar outra quando necessário.

prevalidacao futura

O futuro domínio de pré-validação será dono do processo associado ao espelho da proposta do usuário. Ele coordenará outros domínios por suas portas, mas está fora do escopo inicial.

Cobertura frente a especificacao de pre-validacao

A fonte funcional de comparacao e `api-integracao-mtr-pre-validacao-v1.md` . Os paths da coluna "Endpoint MTR da especificacao" identificam APIs do sistema upstream; eles nao sao rotas publicas do Hub. Neste inventario:

- **IMPLEMENTADO** significa que existem no Hub endpoint publico, porta/caso de uso, adapter MTR e adapter de simulador para a operacao;
- **NAO IMPLEMENTADO** significa que o Hub nao possui Resource, rota publica, capacidade, REST Client, adapter nem simulador para a operacao. Isso nao afirma que a API upstream do MTR nao exista.

Endpoint MTR da especificacao	Estado no Hub	
GET /simtr-parametrizacao/v2/patriarca/processo/identificador-negocial/{identificador}	IMPLEMENTADO	GET /simtr-hub/v1/
GET /simtr-parametrizacao/v1/cadastro/checklist/identificador-negocial/{identificador}/versao/{versao}	IMPLEMENTADO	GET /simtr-hub/v1/
POST /simtr-dossie-produto/v1/dossie-produto	IMPLEMENTADO	POST /simtr-hub/v1/
PATCH /simtr-dossie-produto/v1/dossie-produto/{id}/formulario	IMPLEMENTADO	PATCH /simtr-hub/v1/
POST /simtr-dossie-produto/v2/dossie-produto/{id}/documento	IMPLEMENTADO	POST /simtr-hub/v1/
PATCH /simtr-dossie-produto/v1/dossie-produto/{id}/validacao-negocial	IMPLEMENTADO	PATCH /simtr-hub/v1/
POST /simtr-dossie-produto/v1/dossie-produto/{id}/workflow	IMPLEMENTADO	POST /simtr-hub/v1/
POST /simtr-gestao-documento/v1/storage/container/credencial	IMPLEMENTADO	POST /simtr-hub/v1/
PATCH /simtr-dossie-produto/v1/dossie-produto/{id}/garantia	NAO IMPLEMENTADO	Nao existe
PATCH /simtr-dossie-produto/v1/dossie-produto/{id}/produto	NAO IMPLEMENTADO	Nao existe
POST /simtr-dossie-produto/v1/dossie-produto/{id}/capturar	NAO IMPLEMENTADO	Nao existe
POST /simtr-dossie-produto/v1/dossie-produto/{id}/cancelar	NAO IMPLEMENTADO	Nao existe
GET /simtr-dossie-produto/v2/dossie-produto/{id}	NAO IMPLEMENTADO	Nao existe

Os cinco endpoints ausentes aparecem na especificacao como operacoes existentes do ciclo de vida do dossie que deveriam ser mantidas inalteradas; eles nao aparecem nas duas sequencias principais descritas. As oito operacoes dessas sequencias existem como capacidades atomicas, sem orquestracao: nao existe endpoint unico de pre-validacao nem orquestrador local nesta solucao.

A especificacao usa os nomes de servico `/simtr-parametrizacao` , `/simtr-dossie-produto` e `/simtr-gestao-documento` . Na configuracao atual do Hub, o gateway fornece uma base terminada em `/simtr` e os REST Clients acrescentam, respectivamente, `/parametrizacao` , `/dossie-produto` e `/gestao-documento` . Essa diferenca de montagem nao muda a operacao nem sua versao funcional.

Decisao sobre parametrizacao

`parametrizacao` nao permanece como dominio interno porque descreve o sistema upstream, e nao uma responsabilidade de negocio do Hub.

- A consulta de processo pertence a `arvoredocumento` .
- A consulta de checklist pertence a `conformidade` .
- Os dois podem integrar com APIs do MTR chamadas `parametrizacao` .
- DTOs, mapeadores e modelos nao sao compartilhados automaticamente entre os dois consumidores.

- Duplicacao pequena na borda e preferivel a um acoplamento que impeca a extracao futura.

Camadas por dominio

Estrutura alvo, adaptada apenas quando uma capacidade realmente precisar de cada pasta:

```
<dominio>/
|-- dominio/
|   |-- modelo/
|   |-- erro/
|-- aplicacao/
|   |-- porta/
|       |-- entrada/
|       |-- saida/
|   |-- casodeuso/
|-- adaptador/
|   |-- entrada/
|       |-- rest/v1/
|           |-- dto/
|           |-- mapper/
|           |-- resource/
|-- saida/
|   |-- mtr/
|       |-- dto/<versao>/<operacao>/
|       |-- mapper/
|       |-- client/
|       |-- erro/
|       |-- adapter/
|-- simulador/
|   |-- dto/
|   |-- mapper/
|   |-- adapter/
```

Nao se criam pastas vazias nem abstracoes sem consumidor real.

Regras de dependencia

1. dominio pode usar Java, tipos do proprio dominio e APIs do Quarkus ou de outros frameworks; ele nao depende de aplicacao, Resources, adapters, fachadas ou mapeamentos de borda.
2. aplicacao pode usar tipos do proprio dominio, suas portas e APIs do Quarkus ou de outros frameworks, incluindo Mutiny uni; ela nao importa adapters, DTOs de borda, REST Clients ou outro dominio.
3. Adapters de entrada dependem de portas de entrada e mapeiam DTOs para tipos internos.
4. Adapters de saida implementam portas de saida e mapeiam tipos internos para seus contratos.
5. DTO publico REST, DTO MCP, DTO MTR e DTO do simulador sao contratos independentes.
6. Nenhum mapper converte diretamente DTO publico REST ou DTO MCP em DTO MTR.
7. arquitetura nao importa dominios e nao possui regra, modelo ou erro especifico de negocio.
8. Um adapter local entre dominios fica na borda do consumidor; ele pode importar a API de aplicacao publica do fornecedor e realiza a traducao entre modelos.
9. Um adapter MCP pode depender somente de sua propria borda, das portas de entrada, dos tipos internos referenciados por elas e das APIs tecnicas necessarias ao protocolo. Ele nao depende de adapter REST, adapter MTR, simulador, REST Client nem DTO de outra borda.

A **API publica de aplicacao** de um dominio e formada exclusivamente por suas portas de entrada e pelos tipos semanticos de comando/resultado referenciados por elas. Casos de uso concretos, portas de saida e demais tipos internos nao fazem parte dessa API. A aplicacao de um dominio consumidor nao importa diretamente nem mesmo essa API: somente seu adapter local anticorruptao pode importa-la.

Essas regras estao codificadas no build por ArchUnit para todas as capacidades migradas. ArchUnit protege as fronteiras estruturais e nao mantem blacklist de Quarkus, Jakarta, MicroProfile,

Mutiny, Jackson ou OpenTelemetry por camada. A ativacao ocorreu progressivamente durante a refatoracao para evitar um big bang. Quando existir um adapter MCP, as regras devem tambem provar que dominio e aplicacao nao dependem dele e que ele nao atravessa lateralmente para outros adapters.

Portas e granularidade

- Uma porta de entrada por capacidade atomica.
- Casos de uso expressam verbos de negocio e retornam tipos internos.
- Portas de saida representam a necessidade do consumidor, nao a API generica do fornecedor.
- Um mesmo adapter MTR pode implementar varias portas pequenas do mesmo dominio.
- Uma tool MCP atomica aciona uma porta de entrada; ela nao cria uma segunda representacao da capacidade na aplicacao.
- Nao existe `Service` generico que exponha todas as operacoes de todos os contextos.

Essa granularidade permite que o futuro orquestrador componha somente as capacidades necessarias e que um futuro adapter MCP exponha apenas as tools autorizadas. Ela impede que um consumidor dependa de uma interface ampla por conveniencia.

Modelos internos e nomes

- Records podem ser usados, mas um record nao e automaticamente um value object imutavel.
- Tipos internos recebem nomes semanticos: comandos, resultados, identificadores e conceitos do negocio.
- O sufixo generico `Vo` nao e obrigatorio e sera removido durante a migracao de cada capacidade.
- Comandos podem usar o sufixo `Command`; respostas internas devem representar o resultado, nao o protocolo que as transportou.
- Colecoes continuam com a semantica atual nesta refatoracao. Imutabilidade profunda e tratamento de elementos nulos so mudam mediante decisao funcional e testes proprios.

Contratos das bordas

REST publico

- DTOs de negocio ficam exclusivamente no adapter REST de entrada do proprio dominio.
- DTOs de topo usam nomes explicitos de request e response.
- Tipos aninhados usam nomes semanticos dentro do package da operacao.
- Paths, verbos, status, JSON e validacoes observaveis permanecem iguais.
- O OpenAPI e gerado exclusivamente pelo Quarkus a partir do codigo; nao existe arquivo estatico, filtro, complemento nem teste do documento gerado.
- A unica excecao compartilhada permitida e o contrato tecnico de erro REST, localizado em `arquitetura.excecao.dto`. Ele e proibido no dominio, na aplicacao e nos adapters MCP, MTR e simulador.

MCP futuro

- DTOs, schemas, nomes de tools e erros ficam exclusivamente no adapter MCP do proprio dominio.
- O adapter mapeia entrada MCP para comandos internos e resultados internos para saidas MCP.
- A tool aciona a porta de entrada; nao chama Resource REST, REST Client nem adapter de saida.
- DTOs REST, MTR e do simulador nao sao reutilizados.
- Autenticacao, autorizacao por capacidade, transporte e serializacao permanecem nessa borda.
- A introducao de MCP nao altera paths, verbos, status, JSON ou validacoes observaveis da API REST.

MTR

- DTOs ficam exclusivamente no adapter MTR.
- Sao separados por versao e operacao quando o contrato externo puder evoluir de modo independente.

- REST Clients e annotations de fault tolerance ficam nessa borda.

Simulador

- Implementa as mesmas portas de saída implementadas pelo adapter MTR.
- Trabalha com tipos internos nas portas.
- Quando lê JSON, usa `JSON -> DTO do simulador -> modelo interno`.
- Não reutiliza DTO público nem DTO MTR.
- A seleção MTR/simulador usa qualifiers ou producer CDI explícitos; não pode haver injeção ambígua nem roteador que se injete recursivamente.
- As propriedades atuais de habilitação são preservadas.

Erros

- Exceções e tipos de protocolo HTTP ou MCP não atravessam portas.
- Cada domínio classifica falhas relevantes para seus casos de uso, como rejeição de negócio, dependência indisponível, timeout e resposta externa inválida.
- O adapter MTR traduz erros externos para falhas internas da aplicação.
- A classificação protocolar usada por retry/circuit breaker ocorre dentro da chamada interceptada do REST Client; a tradução para falha interna ocorre somente depois que a política de fault tolerance termina. Essa ordem preserva `retryOn`, `abortOn`, `failOn` e `skipOn` atuais.
- A tradução preserva, sem perdas, todos os dados necessários para reproduzir o contrato público atual: status, recurso, identificador, código, mensagens, detalhe e demais campos observáveis.
- O DTO de erro recebido do MTR é diferente do DTO de erro devolvido pela API pública.
- O adapter REST traduz falhas internas para o status e corpo atuais.
- Um futuro adapter MCP traduz falhas internas para seu próprio resultado estruturado e seguro, sem expor stack, URL interna, token, DTO MTR nem estado do circuit breaker.
- Erros de validação e desserialização, que podem ocorrer antes do método Resource, continuam sob mappers técnicos do adapter REST.

O contrato técnico de erro REST compartilhado e a execução formal a regra geral de DTO por domínio. ArchUnit deve codificar o package permitido e proibir seu uso nas demais bordas, evitando allowlist informal.

Reatividade e chamadas bloqueantes

- `Uni` é usado na camada de aplicação para representar uma operação assíncrona com zero ou um resultado; isso é uma escolha atual, não a única API de framework permitida no núcleo.
- Nenhum caso de uso chama `await`, bloqueia event loop ou cria thread por conta própria.
- Adapters bloqueantes devem deslocar a execução para worker thread sem expor esse detalhe ao domínio.

Fault tolerance e idempotência

As políticas atuais de timeout, retry e circuit breaker devem ser preservadas durante a refatoração, inclusive sua classificação de exceções. Elas pertencem ao adapter MTR e não são aplicadas automaticamente ao simulador.

Ha um risco conhecido: criação de dossiê, inclusão de documento e avanço de workflow são operações mutáveis com retry. Um timeout depois de sucesso remoto pode duplicar efeitos.

Checkpoint bloqueante para workflows e tools mutáveis futuros: antes de um orquestrador ou agente por MCP usar essas operações, deve existir evidência de idempotência do MTR ou uma chave/estratégia idempotente. Essa correção comportamental não será misturada a refatoração estrutural atual.

Observabilidade

Nomes de spans, eventos de log e atributos atuais são contrato de compatibilidade nesta etapa.

- Um inventário e testes de caracterização precedem renomes estruturais.
- A mudança de nomes Java não pode alterar silenciosamente atributos obtidos por reflexão do REST Client.
- Cada capacidade preserva os sinais atuais nas bordas REST, caso de uso e adapter MTR.
- Um futuro adapter MCP identifica a origem da entrada e o nome da tool, preserva a correlação até o MTR e não registra argumentos sensíveis integralmente.
- Redução de logs duplicados e uma mudança posterior, com decisão e testes separados.

Estratégia de testes

Foi usado TDD leve:

1. caracterizar o comportamento observável que ainda não possui teste;
2. provar que o teste protege a fronteira ou falha arquitetural pretendida;
3. migrar a menor fatia vertical possível;
4. executar os testes focados;
5. executar a suíte completa no checkpoint.

Os testes devem cobrir, conforme a capacidade:

- path, verbo, status e JSON público;
- mensagens e caminhos de Jakarta Validation;
- mapeamento REST -> interno -> MTR e caminho inverso;
- JSON realmente enviado e recebido pelo REST Client;
- headers e providers relevantes;
- simulador habilitado e desabilitado;
- tradução de erros ponta a ponta;
- matriz de retry, circuit breaker e timeout sem alterar os valores atuais;
- nomes e atributos de observabilidade considerados contratuais;
- annotations e contratos Java que alimentam o OpenAPI gerado, sem testar ou manipular o artefato;
- propriedades, defaults e perfis de configuração usados pela capacidade;
- regras ArchUnit das capacidades já migradas;
- quando existir MCP, schema e contrato da tool, mapeamento MCP -> interno -> MCP, autenticação, autorização, tradução de erros, propagação do trace e ausência de dados sensíveis nos sinais.

O baseline anterior foi preservado por manifestos e contratos executáveis por capacidade. A evidência quantitativa de cobertura permanece exclusivamente no relatório JaCoCo. Nenhum teste pré-existente foi removido, desabilitado ou substituído sem justificativa e GO humano.

Migração progressiva concluída

A unidade de migração é uma capacidade vertical, não um package inteiro.

Ordem executada:

1. guardrails e caracterização;
2. piloto `IniciarOuAvancarWorkflowDossieProduto`, por possuir contrato pequeno;
3. demais capacidades de `dossieproduto`, uma por vez;
4. `ConsultarProcessoParametrizado` para `arvoredocumento`;
5. `ConsultarChecklist` para `conformidade`;
6. `ObterCredencialContainer` em `gestaodocumento`;
7. remoção segura do antigo package `parametrizacao` e consolidação final.

Cada checkpoint exigiu suíte verde, build válido, regras arquiteturais da fatia ativas e revisão do diff antes da continuidade.

Fora de escopo

- os cinco endpoints marcados como **NAO IMPLEMENTADO** acima e quaisquer outros endpoints novos;
- implementacao de qualquer workflow ou orquestrador;
- escolha definitiva ou dependencia de Quarkus Flow;
- implementacao de MCP Server, tools ou agente consumidor;
- escolha de extensao, versao, transporte ou topologia de implantacao MCP;
- Redis, Cosmos DB ou persistencia de estado de workflow;
- calculo da arvore documental e servicos inteligentes de IA;
- analise de conformidade;
- upload para Azure Blob Storage;
- cache ou renovacao de SAS no Hub;
- mudanca de contratos publicos;
- correcao de idempotencia das operacoes MTR;
- redesenho funcional de nulabilidade ou imutabilidade profunda.

Alternativas rejeitadas

Manter parametrizacao como dominio compartilhado

Rejeitada porque acopla `arvoredocumento` e `conformidade` a um modelo grande definido pelo nome do sistema externo, dificultando evolucao e extracao independentes.

Orquestradores chamarem endpoints REST locais

Rejeitada porque adiciona protocolo, serializacao e falhas de rede dentro do mesmo processo. REST e orquestradores sao entradas para os mesmos casos de uso.

Acoplar MCP ao dominio ou a aplicacao

Rejeitada porque uma tecnologia de entrada nao deve definir a capacidade de negocio. APIs, annotations, DTOs, schemas e erros MCP ficam no adapter; as portas e os casos de uso permanecem independentes do protocolo.

Tools MCP chamarem Resources REST locais

Rejeitada porque cria acoplamento lateral entre adapters, duplica validacao e serializacao e introduz semantica HTTP desnecessaria dentro do mesmo processo. REST e MCP convergem diretamente para a mesma porta de entrada.

Criar um servico MCP separado desde ja

Rejeitada sem necessidade operacional concreta porque adiciona deployment, rede, autenticacao e falhas distribuidas. A separacao permanece uma opcao futura; nesse caso, o servico sera consumidor externo dos contratos publicos do Hub.

Reutilizar os mesmos DTOs em todas as bordas

Rejeitada porque transforma mudancas do MTR em mudancas da API publica e impede o nucleo de evoluir independentemente.

Criar desde ja aggregates e maquinas de estado

Rejeitada porque as regras completas dos fluxos nao fazem parte do escopo. Modelagem inventada criaria falsa complexidade de dominio.

Adicionar Quarkus Flow agora

Rejeitada nesta etapa. O nucleo deve permitir uma avaliacao futura, mas a decisao depende de spike, criterios de operacao e maturidade dos providers de persistencia.

Riscos conhecidos e controles

Risco	Controle
Retry de operacao mutavel duplicar efeito	Preservar agora; bloquear workflow futuro ate validar idempotencia
Alterar erro publico ao separar DTOs	Modelo interno lossless e testes ponta a ponta
Quebrar simulador ao remover Jackson dos modelos	DTO e mapper exclusivos do simulador
Injecao CDI ambigua entre MTR, simulador e roteador	Qualifiers/producer explicitos e teste de bootstrap
Renome quebrar dashboards	Inventario e testes de spans, eventos e atributos
Testes validarem apenas simulador	Stub MTR com simulador desabilitado
Big bang de packages	Uma capacidade vertical por etapa e checkpoints GO/NO-GO
Dependencia MCP vaziar para o nucleo ou outros adapters	Package exclusivo e regras ArchUnit de direcao
Agente repetir ou ordenar incorretamente operacoes mutaveis	Precondicoes no caso de uso, autorizacao por tool e checkpoint de idempotencia
Tool expor credencial ou dado sensivel	Decisao de exposicao separada, menor privilegio e sanitizacao de respostas, logs e traces

Fontes oficiais de framework

- Mutiny e Uni : <https://quarkus.io/guides/mutiny-primer>
- Quarkus REST reativo: <https://quarkus.io/guides/rest>
- SmallRye Fault Tolerance: <https://quarkus.io/guides/smallrye-fault-tolerance>
- Quarkus Flow: <https://quarkus.io/extensions/io.quarkiverse.flow/quarkus-flow/>
- Persistencia do Quarkus Flow: <https://docs.quarkiverse.io/quarkus-flow/dev/persistence.html>

Quarkus Flow 0.12.0 foi publicado para Quarkus 3.33.2.1 , versao atual do projeto, mas a propria documentacao informa que os providers de persistencia ainda passam por testes e benchmarks de alto throughput. Isso reforca que a adocao futura exige spike e nao faz parte desta refatoracao.